Ahmedabad University

Subject - Digital Image Processing

8. Vehicle Orientation Estimation Using Conventional Image Processing

Weekly Report - 3

Date of Submission: 10/10/2026

Repo Link: https://github.com/encrypted-official/DIP_2026_Project8_Group1

Group - 1

Enrollment No.	Name of the Student	Name of the Program
AU2440150	Vehan Gajjar	Btech CSE
AU2340012	Vedant Solanki	Btech CSE
AU2440173	Abhi Jain	Btech CSE
AU2440242	Krish Prajapati	Btech CSE

2026 (Monsoon Semester)

1. Work done this week

This week we moved from reading papers to actually writing code. Our main aim was to get a small working pipeline running from start to end on the DRASHTI-HaOBB dataset, even if the accuracy is not good yet. We split the work into four parts so that each of us had one clear piece to own, and we used the same GitHub repo for everything.

Dataset parsing and crop extraction: We first opened the label files to see how the annotations are stored. The first line of each file holds the flight height, and every line after that has the four corner points of the oriented box, the vehicle class, a difficulty flag and the heading angle. We wrote a parser (parse_annotations.py) that skips the height line and accepts both commas and spaces between values, since we were not fully sure which one the released files use. Then extract_crops.py takes the four corners, makes an axis-aligned box around them, adds 10% padding and saves the crop. We did not rotate the crops upright on purpose, because that would remove the very angle we are trying to find. All crop details and the ground truth go into one metadata.csv file. Vehicles with difficulty 1 (cut off at the image edge) are skipped for now.

Angle convention: Last week one of our doubts was the difference between orientation and heading. We settled it this week. The dataset gives the heading of the vehicle front from 0 to 359 degrees, but methods like PCA and Hough only give an axis, and they cannot tell the front from the back. So we fold the heading into an undirected axis between 0 and 90 degrees using r = heading mod 180 and axis = min(r, 180 - r). For example, 137 degrees becomes 43 and 225 becomes 45. The same rule is used for both the ground truth and the predictions. The angle helpers are in angle_utils.py and we added small unit tests for them (tests/test_angles.py), which pass.

Preprocessing: We kept preprocessing simple and shared between methods (preprocess.py). The crop is converted to grayscale, smoothed with a 3x3 Gaussian blur, enhanced with CLAHE and then passed to Canny edge detection with thresholds 50 and 150.

Method A, PCA on edge pixels: estimate_pca.py takes the coordinates of all edge pixels, finds their covariance matrix and uses the eigenvector with the largest eigenvalue as the vehicle axis. If there are fewer than 20 edge pixels, or the point cloud is almost round (ratio of eigenvalues below 1.05), the method returns no prediction instead of a random angle.

Method B, Hough lines: estimate_edges.py runs the probabilistic Hough transform (HoughLinesP) on the same edge map, removes short segments and then combines the remaining line angles. Longer lines are given more weight. Since 10 degrees and 190 degrees are the same axis, we average the doubled angles and then halve the result.

Evaluation and visualisation: evaluate.py joins the predictions with the metadata and computes the shortest angular error between axes. It reports MAE, median error, 90th percentile error, accuracy within 5, 10 and 15 degrees, and coverage (how many crops actually got a prediction). Invalid predictions are counted and not silently removed, so a method cannot look better just by

skipping hard crops. It also gives a class-wise table. visualize_results.py draws the ground truth axis and the predicted axis on the crop so we can check failures by eye.

We also wrote short notes in the docs folder about the dataset format, the comparison between the two methods, a common Git workflow, and a template for the analysis. The scripts are made to run first on 10 validation images so that everyone can test quickly before using a bigger set.

2. Work to be done next week

Now that the basic pipeline is in place, next week is mostly about running it properly and understanding the results. We plan to:

- Run both methods on a larger part of the validation split (around 50 to 200 vehicle crops) and fill in the results table with real numbers.
- Look at the overlay images and pick out cases where each method works and where it clearly fails, and try to find the reason (shadows, road lines, very small crops, etc.).
- Check the results class-wise and by crop size, but only where we have enough samples to say something useful.
- Add a third method using image moments or a minimum-area rectangle on a foreground mask, so we can compare it with PCA and Hough.
- Try a few preprocessing changes, such as different Canny thresholds or turning CLAHE off, and see how much the error changes.
- Double check the angle convention of the dataset against a few crops by hand, since image y-axis points downward and that could flip angles.

3. Literature Review

This week our reading was more focused on the exact techniques we used in the code, so that we understand why each step works and where it can go wrong.

- Canny (1986) proposed the edge detector that we use as the first step of both methods. He defined three goals for a good detector: low error rate, good localisation and only one response per edge. The two thresholds with hysteresis explain why our edge map changes a lot when we change the 50 and 150 values.
- Duda and Hart (1972) introduced the rho-theta form of the Hough transform for finding straight lines. Matas, Galambos and Kittler (2000) later gave the progressive probabilistic Hough transform, which is what OpenCV's HoughLinesP is based on. It returns line segments with end points, which is why we could weight each line by its length.
- Zuiderveld (1994) described CLAHE, where histogram equalisation is done on small tiles and the contrast is clipped so that noise does not get boosted too much. We used it because many vehicles in aerial images have weak contrast with the road.

- Hu (1962) worked on image moments and moment invariants for shape recognition. The second-order central moments give the orientation of a shape, and this is basically the same idea as our PCA step. Because of this we understood that PCA and image moments should not be shown as two fully separate methods.
- Mardia and Jupp (2000) in their book on directional statistics explain how to average axial data, where an angle and the angle plus 180 degrees mean the same thing. The normal mean gives wrong answers here, and the fix is to double the angles first. We used this directly when combining Hough line directions.
- Xia et al. (2018) built the DOTA dataset for object detection in aerial images, which uses oriented bounding boxes given by four corner points. Reading it helped us understand the OBB format in DRASHTI-HaOBB and why axis-aligned boxes alone are not enough for rotated vehicles.

4. Challenges Faced

- Choosing the right target. The dataset gives the full heading, but our methods can only give an axis. It took some discussion before we agreed to fold everything to 0 to 90 degrees and to compare angles only after that.
- Not being fully sure about the label format, for example whether values are comma or space separated and how the first line should be handled. We made the parser accept both and print a message for rows it cannot read.
- Avoiding data leakage. The box corners are used to cut the crop and get the ground truth, so we had to make sure the estimators only see the crop pixels and never the annotation angle or box size.
- Edge maps are noisy. Road markings, shadows, windows and nearby vehicles also produce edges, and these can pull the PCA axis or the Hough lines in the wrong direction.
- Some crops are very small or have no long straight edge, so Hough finds no line and PCA sometimes gets an almost round point cloud. We had to decide how to handle these cases and chose to mark them as no prediction and report coverage separately.
- Making four people's code fit together. The PCA and Hough scripts, the evaluation and the plots all depend on the same CSV columns, so we had to fix the column names early and agree that each person commits only their own files.

References

[1] J. Canny, "A Computational Approach to Edge Detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679-698, 1986.

[2] R. O. Duda and P. E. Hart, "Use of the Hough Transformation to Detect Lines and Curves in Pictures," Communications of the ACM, vol. 15, no. 1, pp. 11-15, 1972.

[3] J. Matas, C. Galambos and J. Kittler, "Robust Detection of Lines Using the Progressive Probabilistic Hough Transform," Computer Vision and Image Understanding, vol. 78, no. 1, pp. 119-137, 2000.

[4] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," in Graphics Gems IV, Academic Press, pp. 474-485, 1994.

[5] M.-K. Hu, "Visual Pattern Recognition by Moment Invariants," IRE Transactions on Information Theory, vol. 8, no. 2, pp. 179-187, 1962.

[6] K. V. Mardia and P. E. Jupp, Directional Statistics, John Wiley and Sons, 2000.

[7] G.-S. Xia et al., "DOTA: A Large-Scale Dataset for Object Detection in Aerial Images," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3974-3983, 2018.

[8] DRASHTI-HaOBB dataset, Zenodo, https://zenodo.org/records/18278989